

Department of Information Engineering Technology
The Superior University, Lahore

Mobile Application Development Lab

Experiment No.8

Navigator, Hero Animation, BottomNavBar

Prepared for

Eisha Nawaz

By:

Name: _____

ID: _____

Section: _____

Semester: _____

Total Marks: 10

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.8

Navigator, Hero Animation, BottomNavBar (Rubrics)

Name: _____

Roll No: _____

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		6	Sub Total marks Obtained in Psychomotor(P)					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		4	Sub Total marks Obtained in Affective (A)					

Instructor Name: Eisha Nawaz

Total Marks (P+A): 10

Instructor Signature: _____

Obtained Marks (P+A): _____

Navigator, Hero Animation, BottomNavBar

1.1 Objectives:

The objective of this lab is to master the widgets of flutter and to learn how to use it.

1.2 Overview:

This lab is designed to teach the skills needed to become proficient with the primary framework: flutter.

1.3 Materials and Equipment

- A lab PC or laptop
- Flutter framework

Animation while navigating to different pages is also important if it helps to convey an action, rather than simply being a distraction. What does this mean? Just because you can show fancy animations does not mean you should. Use animations to enhance the UX, not frustrate the user.

USING THE NAVIGATOR

The `Navigator` widget manages a stack of *routes* to move between pages. You can optionally pass data to the destination page and back to the original page. To start navigating between pages, you use the `Navigator.push`, `pushNamed`, and `pop` methods. (You'll learn how to use the `pushNamed` method in the "Using the Named Navigator Route" section of this chapter.) `Navigator` is incredibly smart; it shows native navigation on iOS or Android. For example, in iOS when navigating to a new page, you usually slide the next page from the right side of the screen toward the left. In Android when navigating to a new page, you typically slide the next page from the bottom of the screen toward the top. To summarize, in iOS, the new page slides in from the right, and in Android, it slides in from the bottom.

The following example shows you how to use the `Navigator.push` method to navigate to the About page. The `push` method passes the `BuildContext` and `Route` arguments. To push a new `Route` argument, you create an instance of the `MaterialPageRoute` class that replaces the screen with the appropriate platform (iOS or Android) animation transition. In the example, the `fullscreenDialog` property is set to `true` to present the About page as a full-screen modal dialog. By setting the `fullscreenDialog` property to `true`, the About page app bar automatically includes a close button. In iOS, the modal dialog transition presents the page by sliding from the bottom of the screen toward the top, and this is also the default for Android.

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    fullscreenDialog: true,  
    builder: (context) => About(),  
  ),  
);
```

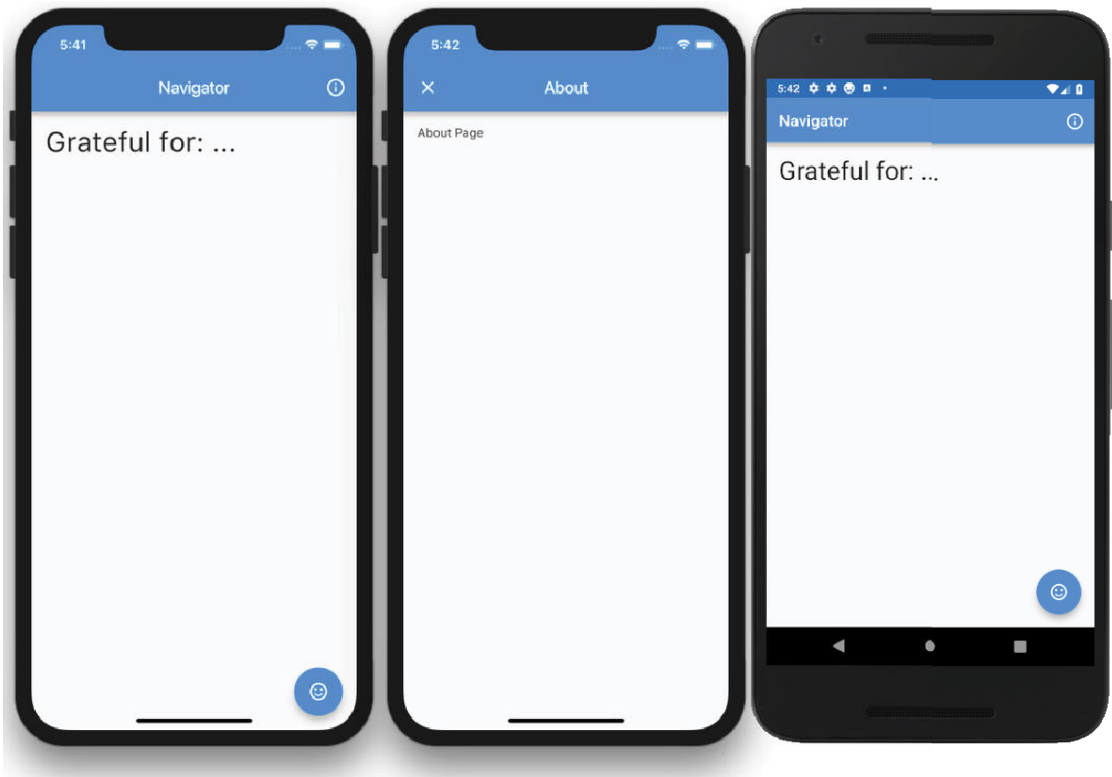
The following example shows how to use the `Navigator.pop` method to close the page and navigate back to the previous page. You call the `Navigator.pop(context)` method by passing the `BuildContext` argument, and the page closes by sliding from the top of the screen toward the bottom. The second example shows how to pass a value back to the previous page.

```
// Close page
Navigator.pop(context);

// Close page and pass a value back to previous page
Navigator.pop(context, 'Done');
```

TRY IT OUT Creating the Navigator App, Part 1—The About Page

The project has a main home page with a `FloatingActionButton` to navigate to a gratitude page by passing a default selected `Radio` button value. The gratitude page shows three `Radio` buttons to select a value and then pass it back to the home page and update the `Text` widget with the appropriate value. The `AppBar` has an actions `IconButton` that navigates to the about page by passing a `fullScreenDialog` argument set to `true` to create a full-screen modal dialog. The modal dialog shows a close button at the upper left of the page and animates from the bottom. In this first part, you'll develop the navigation from the main page to the About page and back.



1. Create a new Flutter project and name it `ch8_navigator`. Refer to the instructions in Chapter 4, “Creating a Starter Project Template.” For this project, you need to create only the `pages` folder.
2. Open the `home.dart` file and add an `IconButton` to the `AppBar` actions `Widget` list. The `IconButton` `onPressed` property will call the method `_openPageAbout()` and pass `context` and `fullScreenDialog` arguments. Do not worry about the squiggly red lines under the method name; you'll create that method in later steps. The `context` argument is needed by the `Navigator` widget, and the `fullScreenDialog` argument is set to `true` to show the About page as a full-screen modal. If you set the `fullScreenDialog` argument to `false`, the About page shows a back arrow instead of a close button icon.

```

    appBar: AppBar(
      title: Text('Navigator'),
      actions: <Widget>[
        IconButton(
          icon: Icon(Icons.info_outline),
          onPressed: () => _openPageAbout(
            context: context,
            fullscreenDialog: true,
          ),
        ),
      ],
    ),
  ),
),

```

- ### 3. Add a SafeArea with Padding as a child to the body.

```
body: SafeArea(
  child: Padding(),
),
```

4. In the `PaddingChild`, add a `Text()` widget, with the text `'Grateful for: $_howAreYou'`. Notice the `$_howAreYou` variable, which will contain the returned value when navigating back (`Navigator.pop`) from the gratitude page. Add a `TextStyle` class with a `fontSize` value of 32.0 pixels.

```
body: SafeArea(
  child: Padding(
    padding: EdgeInsets.all(16.0),
    child: Text('Grateful for: $_howAreYou', style: TextStyle(fontSize: 32.0)),
  ),
),
```

5. To the Scaffold `floatingActionButton` property, add a `FloatingActionButton()` widget with an `onPressed` that calls the method `_openPageGratitude(context: context)`, which passes the `context` argument.

- For the `FloatingActionButton()` child, add the icon called `sentiment_satisfied`, and for the tooltip, add the 'About' description.

```
floatingActionButton: FloatingActionButton(
  onPressed: () => _openPageGratitude(context: context),
  tooltip: 'About',
  child: Icon(Icons.sentiment_satisfied),
),
```

- On the first line after the `_HomeState` class definition, add a `String` variable called `_howAreYou` with a default value of `'...'`.

```
String    howAreYou = "...";
```

- Continue by adding the `_openPageAbout()` method that accepts `BuildContext` and `bool` named parameters with the default values set to `false`.

```
void _openPageAbout({BuildContext context, bool fullscreenDialog = false}) {}
```

9. In the `openPageAbout()` method, add a `Navigator.push()` method with a context and a second argument of `MaterialPageRoute()`. The `MaterialPageRoute()` class passes the `fullscreenDialog` argument and a builder that calls the `About()` page that you'll create in later steps.

```
void _openPageAbout({BuildContext context, bool fullscreenDialog = false}) {
  Navigator.push(
```

```

        context,
        MaterialPageRoute(
          fullscreenDialog: fullscreenDialog,
          builder: (context) => About(),
        ),
      );
    }
}

```

10. On top of the `home.dart` page, import the `about.dart` page that you'll create next.

```
import 'about.dart';
```

`Navigator` is simple to use but also powerful. Let's examine how it works. `Navigator.push()` passes two arguments: `context` and `MaterialPageRoute`. For the first argument, you pass the `context` argument. The second argument, `MaterialPageRoute()`, gives you the horsepower to navigate to another page using platform-specific animation. Only the builder is required to navigate with the optional `fullscreenDialog` argument.

11. Create a new file called `about.dart` in the `lib/pages` folder. Since this page only displays information, create a `StatelessWidget` class called `About`.

12. For the body, add the usual `SafeArea` with `Padding` and the `child` property as a `Text` widget.

```

// about.dart
import 'package:flutter/material.dart';

class About extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('About'),
      ),
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Text('About Page'),
        ),
      ),
    );
  }
}

```

How It Works

You add to the `AppBar` an `IconButton` under the `actions` property. The `icon` property for the `IconButton` is set to `Icons.info_outline`, the `_openPageAbout()` method passes the `context`, and the `fullscreenDialog` argument is set to `true`. You also add to the `Scaffold` a `FloatingActionButton` that calls the `_openPageGratitude()` method. The `_openPageAbout()` method uses the `Navigator.push()` method to pass the `context` and the `MaterialPageRoute`. The `MaterialPageRoute` passes the `fullscreenDialog` argument set to `true`, and the builder calls the `About()` page. The `About` page class is a `StatelessWidget` with a `Scaffold` and `AppBar`; the `body` property has a `SafeArea` with `Padding` as a child that shows a `Text` widget with the "About Page" text.

TRY IT OUT Creating the Navigator App, Part 2—The Gratitude Page

The second part of the app is to navigate to the gratitude page by passing a default value to select the appropriate `Radio` button. Once you navigate back to the home page, the newly selected `Radio` button value is passed back and displayed in the `Text` widget.

1. Open the `home.dart` file, and after the `_openPageAbout()` method, add the `_openPageGratitude()` method. The `_openPageGratitude()` method takes two parameters: a `context` and a `fullscreenDialog` `bool` variable with a default value of `false`. In this case, the gratitude page is not a `fullscreenDialog`. Like the previous `MaterialPageRoute`, the `builder` opens the page. In this case, it is the gratitude page.

Note that when passing data to the gratitude page and waiting to receive a response, the method is marked as `async` to wait a response from `Navigator.push` by using the `await` keyword.

```
void _openPageGratitude(  
  {BuildContext context, bool fullscreenDialog = false}) async {  
  final String _gratitudeResponse = await Navigator.push(  

```

The `MaterialPageRoute` `builder` builds the contents of the route. In this case, the content is the gratitude page, which accepts a `radioGroupValue` `int` parameter with a value of `-1`. The `-1` value tells the `Gratitude` class page not to select any `Radio` buttons. If you pass a value like `2`, it selects the appropriate `Radio` button that corresponds to this value.

```
    builder: (context) => Gratitude(  
      radioGroupValue: -1,  
    ),  

```

Once the user dismisses the gratitude page, the `_gratitudeResponse` variable is populated. Use the `??` (double question marks if `null`) operator to check the `_gratitudeResponse` for a valid value (not `null`) and populate the `_howAreYou` variable. The `Text` widget is populated with the `_howAreYou` value on the home page with the appropriate selected gratitude value or an empty string. In other words, if the `_gratitudeResponse` value is not `null` the `_howAreYou` variable is populated with the `_gratitudeResponse` value; otherwise the `_howAreYou` variable is populated by an empty string.

```
_howAreYou = _gratitudeResponse ?? '';
```

Here's the full `_openPageGratitude()` method code:

```
void _openPageGratitude(  
  {BuildContext context, bool fullscreenDialog = false}) async {  
  final String _gratitudeResponse = await Navigator.push(  
    context,  
    MaterialPageRoute(  
      fullscreenDialog: fullscreenDialog,  
      builder: (context) => Gratitude(  
        radioGroupValue: -1,  
      ),  
    ),  

```



```

    ),
  );
  _howAreYou = _gratitudeResponse ?? '';
}

```

2. At the top of the `home.dart` page, add the `gratitude.dart` page that you'll create next.

```
import 'gratitude.dart';
```

3. Create a new file named `gratitude.dart` in the `lib/pages` folder. Since this page will modify data (state), create a `StatefulWidget` class called `Gratitude`.

To receive data passed from the home page, modify the `Gratitude` class by adding a final `int` variable named `radioGroupValue`. Note the final variable does not start with an underscore. Create a named constructor requiring this parameter. The `radioGroupValue` variable is accessed by the class `_GratitudeState` extends `State<Gratitude>` by calling `widget.radioGroupValue`.

```

class Gratitude extends StatefulWidget {
  final int radioGroupValue;

  Gratitude({Key key, @required this.radioGroupValue}) : super(key: key);

  @override
  _GratitudeState createState() => _GratitudeState();
}

```

4. For the Scaffold AppBar, add an `IconButton` to the actions list of `Widget`. Set the `IconButton` icon to `Icons.check` with the `onPressed` property calling the `Navigator.pop`, which returns the `_selectedGratitude` to the home page.

```

appBar: AppBar(
  title: Text('Gratitude'),
  actions: <Widget>[
    IconButton(
      icon: Icon(Icons.check),
      onPressed: () => Navigator.pop(context, _selectedGratitude),
    ),
  ],
),

```

5. For the body, add the usual `SafeArea` and `Padding` with `child` property as a `Row`.

The `Row` children list of `Widget` contains three alternating `Radio` and `Text` widgets. The `Radio` widget takes the `value`, `groupValue`, and `onChanged` properties. The `value` property is the ID value for the `Radio` button. The `groupValue` property holds the value of the currently selected `Radio` button. The `onChanged` passes the selected index value to the custom method `_radioOnChanged()` that handles which `Radio` button is currently selected. Following each `Radio` button, there is a `Text` widget that acts as a label for the `Radio` button.

Here's the full body source code:

```
body: SafeArea(
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Row(
      children: <Widget>[
        Radio(
          value: 0,
          groupValue: _radioGroupValue,
          onChanged: (index) => _radioOnChanged(index),
        ),
        Text('Family'),
        Radio(
          value: 1,
          groupValue: _radioGroupValue,
          onChanged: (index) => _radioOnChanged(index),
        ),
        Text('Friends'),
        Radio(
          value: 2,
          groupValue: _radioGroupValue,
          onChanged: (index) => _radioOnChanged(index),
        ),
        Text('Coffee'),
      ],
    ),
  ),
),
```

6. On the first line after the `_HomeState` class definition, add three variables—`_gratitudeList`, `_selectedGratitude`, and `_radioGroupValue`—and the `_radioOnChanged()` method.

➤ `_gratitudeList` is a List of String values.

```
List<String> _gratitudeList = List();
```

➤ `_selectedGratitude` is a String variable containing the selected Radio button value.

```
String _selectedGratitude;
```

➤ `_radioGroupValue` is an int containing the ID of the selected Radio button value.

```
int _radioGroupValue;
```

7. Create the `_radioOnChanged()` method taking an int for the selected index of the Radio button. In the method, call `setState()` to have the Radio widgets update with the selected value. The `_radioGroupValue` variable is updated with the index. The `_selectedGratitude` variable (example value `Coffee`) is updated by taking the `_gratitudeList[index]` list value by the selected index (position in the list).

```
void _radioOnChanged(int index) {
  setState(() {
    _radioGroupValue = index;
    _selectedGratitude = _gratitudeList[index];
    print('_selectedRadioValue $_selectedGratitude');
  });
}
```

8. Override `initState()` to initialize the `_gratitudeList`. Since the `_radioGroupValue` is passed from the home page, initialize it with `widget.radioGroupValue`, which is the final variable passed from the home page.

```
_gratitudeList..add('Family')..add('Friends')..add('Coffee');  
_radioGroupValue = widget.radioGroupValue;
```

The following is the code that declares all variables and methods:

```
class _GratitudeState extends State<Gratitude> {  
  List<String> _gratitudeList = List();  
  String _selectedGratitude;  
  int _radioGroupValue;  
  
  void _radioOnChanged(int index) {  
    setState(() {  
      _radioGroupValue = index;  
      _selectedGratitude = _gratitudeList[index];  
      print('_selectedRadioValue $_selectedGratitude');  
    });  
  }  
  
  @override  
  void initState() {  
    super.initState();  
  
    _gratitudeList..add('Family')..add('Friends')..add('Coffee');  
    _radioGroupValue = widget.radioGroupValue;  
  }  
}
```

Here's the entire `gratitude.dart` file source code:

```
import 'package:flutter/material.dart';  
  
class Gratitude extends StatefulWidget {  
  final int radioGroupValue;  
  
  Gratitude({Key key, @required this.radioGroupValue}) : super(key: key);  
  
  @override  
  _GratitudeState createState() => _GratitudeState();  
}  
  
class _GratitudeState extends State<Gratitude> {  
  List<String> _gratitudeList = List();  
  String _selectedGratitude;  
  int _radioGroupValue;  
  
  void _radioOnChanged(int index) {  
    setState(() {  
      _radioGroupValue = index;  
      _selectedGratitude = _gratitudeList[index];  
      print('_selectedRadioValue $_selectedGratitude');  
    });  
  }  
}
```

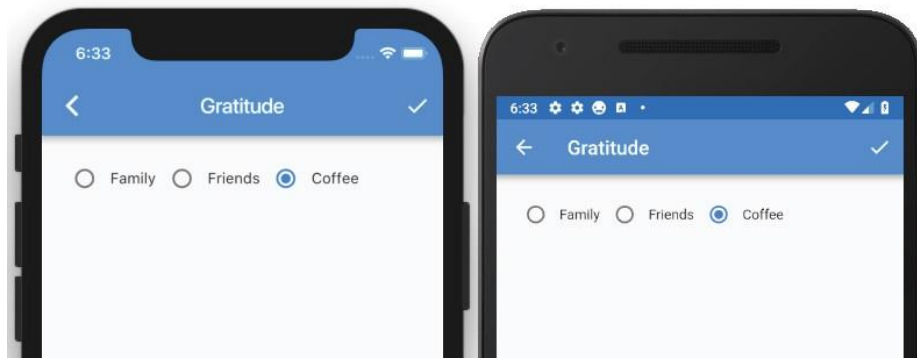
```

@override
void initState() {
  super.initState();

  _gratitudeList..add('Family')..add('Friends')..add('Coffee');
  _radioGroupValue = widget.radioGroupValue;
}

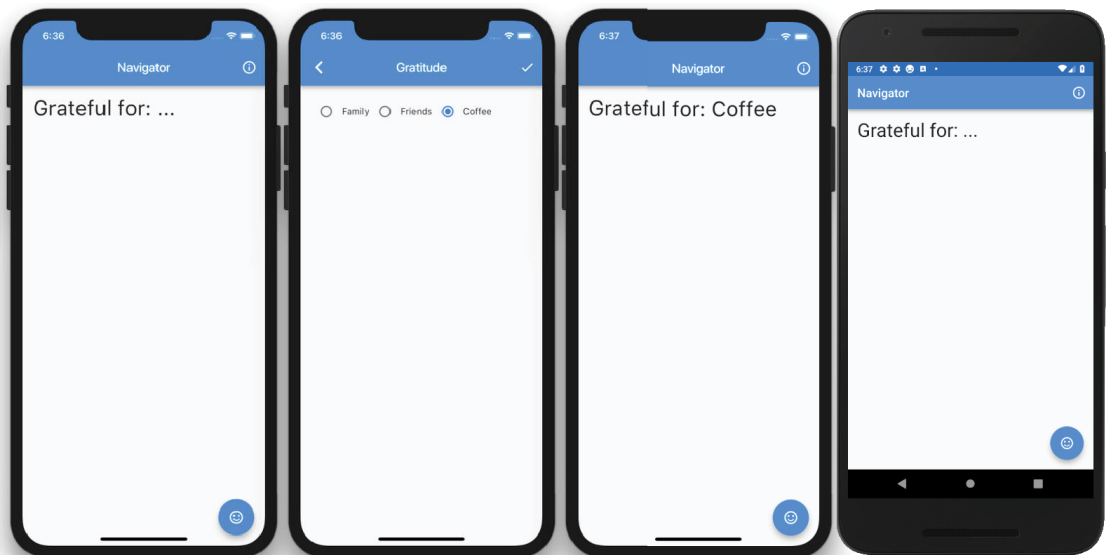
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Gratitude'),
      actions: <Widget>[
        IconButton(
          icon: Icon(Icons.check),
          onPressed: () => Navigator.pop(context, _selectedGratitude),
        ),
      ],
    ),
    body: SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Row(
          children: <Widget>[
            Radio(
              value: 0,
              groupValue: _radioGroupValue,
              onChanged: (index) => _radioOnChanged(index),
            ),
            Text('Family'),
            Radio(
              value: 1,
              groupValue: _radioGroupValue,
              onChanged: (index) => _radioOnChanged(index),
            ),
            Text('Friends'),
            Radio(
              value: 2,
              groupValue: _radioGroupValue,
              onChanged: (index) => _radioOnChanged(index),
            ),
            Text('Coffee'),
          ],
        ),
      ),
    ),
  );
}

```



How It Works

You have the entire app created with a home page that can navigate to the about page as a `full-screenDialog`. The `fullscreenDialog` gives the about page a default close action button. By tapping the home page's `FloatingActionButton`, the `Navigator MaterialPageRoute` builder builds the contents for the route, in this case the gratitude page. Through the `Gratitude` constructor, data is passed to unselected `Radio` buttons. From the gratitude page, a list of `Radio` buttons gives a choice to select a gratitude. By tapping the `AppBar` action button (checkbox `IconButton`), the `Navigator.pop` method passes the selected gratitude value back to the `Home Text` widget. From the home page, you called the `Navigator.push` method by using the `await` keyword, and the method has been waiting to receive a value. Once the `About` page's `Navigator.pop` method is called, it returns a value to the home page's `_gratitudeResponse` variable. Using the `await` keyword is a powerful and straightforward feature to implement.



Using the Named Navigator Route

An alternate way to use `Navigator` is to refer to the page that you are navigating to by the route name. The route name starts with a slash, and then comes the route name. For example, the About page route name is `'/about'` . The list of routes is built into the `MaterialApp()` widget. The routes have a Map of `String` and `WidgetBuilder` where the `String` is the route name, and the `WidgetBuilder` has a builder to build the contents of the route by the `Class` name (About) of the page to open.

```
routes: <String, WidgetBuilder>{  
  '/about': (BuildContext context) => About(),  
  '/gratitude': (BuildContext context) => Gratitude(),  
},
```

To call the route, the `Navigator.pushNamed()` method is called by passing two arguments. The first argument is `context`, and the second is the route name.

```
Navigator.pushNamed(context, '/about');
```

USING HERO ANIMATION

The `Hero` widget is a great out-of-the-box animation to convey the navigation action of a widget flying into place from one page to another. The hero animation is a shared element transition (animation) between two different pages.

To visualize the animation, imagine seeing a superhero flying into action. For example, you have a list of journal entries with a photo thumbnail, the user selects an entry, and you see the photo thumbnail transition to the detail page by moving and growing to full size. The photo thumbnail is the superhero, and when tapped, it flies into action by moving from the list page to the detail page and lands perfectly on the correct location at the top of the detail page showing the full photo. When the detail page is dismissed, the `Hero` widget flies back to the original page, position, and size. In other words, the animation shows the photo thumbnail moving and growing into place from the list page to the detail page, and once the detail page is dismissed, the animation and size are reversed. The `Hero` widget has all of these features built in; there's no need to write custom code to handle the size and animation between pages.

To continue with the previous scenario, you wrap the list page `Image` widget as a child of the `Hero` widget and assign a `tag` property name. Repeat the same steps for the detail page and make sure the `tag` property value is the same in both pages.

```
// List page  
Hero(  
  tag: 'photo1',  
  child: Image(  
    image: AssetImage("assets/images/coffee.png"),  
  ),  
)  
  
// Detail page  
Hero(  
  tag: 'photo1',  
  child: Image(  
    image: AssetImage("assets/images/coffee.png"),  
  ),  
)
```

```

        tag: 'photo1',
        child: Container(
          child: Image(
            image: AssetImage("assets/images/coffee.png"),
          ),
        ),
      ),
    ),
  ),
),

```

The `Hero` child widget is marked for hero animation. When the `Navigator` pushes or pops a `PageRoute`, the entire screen's content is replaced. This means during the animation transition the `Hero` widget is not shown in the original position in both the old and new routes, but it moves and resizes from one page to another. Each `Hero` tag must be unique and match on both the originating and landing pages.

TRY IT OUT Creating the Hero Animation App

In this example, the `Hero` widget has an `Icon` as a child wrapped in a `GestureDetector`. An `InkWell` could also be used instead of the `GestureDetector` to show a material tap animation. The `InkWell` widget is a Material Component that responds to touch gestures by displaying a splash (ripple) effect. You'll take a more in-depth look at `GestureDetector` and `InkWell` in Chapter 11, "Applying Interactivity." When the `Icon` is tapped, `Navigator.push` is called to navigate to the detail screen, called `Fly`. Since the `Hero` widget animation is like a superhero flying, the detail screen is named `Fly`.

1. Create a new Flutter project and name it `ch8_hero_animation`. Again, you can follow the instructions in Chapter 4. For this project, you need to create only the `pages` folder.
2. Open the `home.dart` file and add to the `body` a `SafeArea` with `Padding` as a child.

```

body: SafeArea(
  child: Padding(),
),

```

3. In the `Padding` child, add the `GestureDetector()` widget, which you will create next.

```

body: SafeArea(
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: GestureDetector(),
  ),
),

```

4. Add to the `GestureDetector` child a `Hero` widget with tag `'format_paint'`; tag can be any unique ID. The `Hero` widget child is a `format_paint` icon with `lightGreen` color and a size value of 120.0 pixels. Note you could have used an `InkWell()` widget instead of `GestureDetector()`. The `InkWell()` widget shows a splash feedback where tapped, but the `GestureDetector()` widget will not show the touch feedback. For the `onTap` property, you call `Navigator.push` to open the detail page called `Fly`.

```

GestureDetector(
  child: Hero(
    tag: 'format_paint',

```

```

        child: Icon(
          Icons.format_paint,
          color: Colors.lightGreen,
          size: 120.0,
        ),
      ),
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(builder: (context) => Fly()),
      );
    },
  ),
),

```

5. At the top of the `home.dart` page, import the `fly.dart` page that you will create next.

```
import 'fly.dart';
```

Here's the entire `Home.dart` file source code:

```
import 'package:flutter/material.dart';
import 'fly.dart';

class Home extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Hero Animation'),
      ),
      body: SafeArea(
        child: Padding(
          padding: EdgeInsets.all(16.0),
          child: GestureDetector(
            child: Hero(
              tag: 'format_paint',
              child: Icon(
                Icons.format_paint,
                color: Colors.lightGreen,
                size: 120.0,
              ),
            ),
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => Fly()),
              );
            },
          ),
        ),
      ),
    );
  }
}
```


6. Create a new file called `fly.dart` in the `lib/pages` folder. Since this page only displays information, create a `StatelessWidget` class called `Fly`. For the body, add the usual `SafeArea` with a child set to the `Hero()` widget.

```
body: SafeArea(  
  child: Hero(),  
),
```

7. To calculate the width of the `Icon`, after `Widget build(BuildContext context) {}`, add a double variable called `_width` set by `MediaQuery.of(context).size.shortestSide / 2`. The `shortestSide` property returns the lesser of the width or height of the screen, and you divide by two to make it half the size.

The reason you calculate the width is only to resize the `Icon` width according to device size and orientation. If you used an `Image` instead, this calculation would not be necessary; it can be done by using `BoxFit.fitWidth`.

```
double _width = MediaQuery.of(context).size.shortestSide / 2;
```

8. Add to the `Hero` widget a tag of `'format_paint'` with a `Container` for the child. Note for the `Hero` widget to work properly, the tag needs to be the same name you gave in the `GestureDetector` child `Hero` widget in the `home.dart` file. The `Container` child is a `format_paint` icon with `lightGreen` color and a size value of the width variable. For the `Container` alignment property, use `Alignment.bottomCenter`. You can experiment using different `Alignment` values to see the hero animation variations at work.

```
Hero(  
  tag: 'format_paint',  
  child: Container(  
    alignment: Alignment.bottomCenter,  
    child: Icon(  
      Icons.format_paint,  
      color: Colors.lightGreen,  
      size: _width,  
    ),  
  ),  
)
```

Here's the entire `Fly.dart` file source code:

```
import 'package:flutter/material.dart';  
  
class Fly extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    double _width = MediaQuery.of(context).size.shortestSide / 2;  
  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Fly'),  
      ),  
    ),  
  ),  
}
```

```

body: SafeArea(
  child: Hero(
    tag: 'format_paint',
    child: Container(
      alignment: Alignment.bottomCenter,
      child: Icon(
        Icons.format_paint,
        color: Colors.lightGreen,
        size: _width,
      ),
    ),
  ),
),
);
}

```



How It Works

The hero animation is a powerful built-in animation to convey an action by automatically animating a widget from one page to another to the correct size and position. On the home page, you declare a `GestureDetector` with the `Hero` widget as the child widget. The `Hero` widget sets an `Icon` as the child. The `onTap` calls the `Navigator.push()` method, which navigates to the page `Fly`. All you need to do on the `Fly` page is to declare the widget you're animating to as a child of the `Hero` widget. When you navigate back to the home page, the `Hero` animates the `Icon` to the original position.

USING THE BOTTOMNAVIGATIONBAR

`BottomNavigationBar` is a Material Design widget that displays a list of `BottomNavigationBarItems` that contains an icon and a title at the bottom of the page (Figure 8.1). When the `BottomNavigationBarItem` is selected, the appropriate page is built.

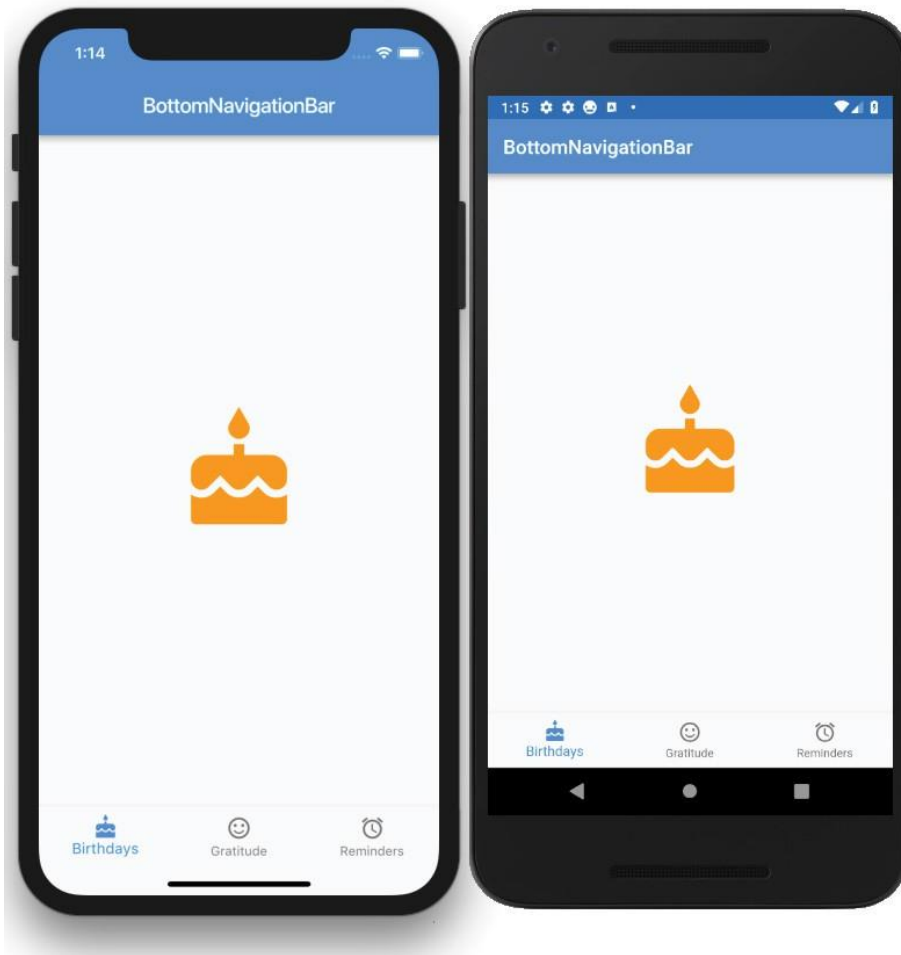


FIGURE 8.1: Final `BottomNavigationBar` with icons and titles

TRY IT OUT Creating the `BottomNavigationBar` App

In this example, `BottomNavigationBar` has three `BottomNavigationBarItems` that replace the current page with the selected one. There are different ways to display the selected page; you use a `Widget` class as a variable.

1. Create a new Flutter project and name it `ch8_bottom_navigation_bar`. As always, you can follow the instructions in Chapter 4. For this project, you need to create only the `pages` folder.

2. Open the `home.dart` file, and add to the `body` a `SafeArea` with `Padding` as a child.

```
body: SafeArea(  
  child: Padding(),  
),
```

3. In the `Padding` child, add the variable `Widget _currentPage`, which you will create next. Note this time you are using a `Widget` class to create the `_currentPage` variable that holds each page that is selected, either the `Gratitude`, `Reminders`, or `Birthdays` `StatelessWidget` class.

```
body: SafeArea(  
  child: Padding(  
    padding: const EdgeInsets.all(16.0),  
    child: _currentPage,  
  ),  
),
```

4. Add to the `Scaffold` `bottomNavigationBar` property a `BottomNavigationBar` widget. For the `currentIndex` property, use the `_currentIndex` variable, which will be created later.

```
bottomNavigationBar: BottomNavigationBar(  
  currentIndex: _currentIndex,
```

The `items` property is a `List` of `BottomNavigationBarItems`. Each `BottomNavigationBarItem` takes an `icon` property and a `title` property.

5. Add to the `items` property three `BottomNavigationBarItems` with icon `cake`, `sentiment_satisfied`, and `access_alarm`. The titles are `'Birthdays'`, `'Gratitude'`, and `'Reminders'`.

```
items: [  
  BottomNavigationBarItem(  
    icon: Icon(Icons.cake),  
    title: Text('Birthdays'),  
  ),
```

6. For the `onTap` property, the callback returns the current index of the active item. Name the variable `selectedIndex`.

```
onTap: (selectedIndex) => _changePage(selectedIndex),
```

Here's the entire `BottomNavigationBar` code:

```
bottomNavigationBar: BottomNavigationBar(  
  currentIndex: _currentIndex,  
  items: [  
    BottomNavigationBarItem(  
      icon: Icon(Icons.cake),  
      title: Text('Birthdays'),  
    ),
```

```

        BottomNavigationBarItem(
          icon: Icon(Icons.sentiment_satisfied),
          title: Text('Gratitude'),
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.access_alarm),
          title: Text('Reminders'),
        ),
      ],
      onTap: (selectedIndex) => _changePage(selectedIndex),
    ),
  ),

```

7. Add the `_changePage(int selectedIndex)` method after `Scaffold()`. The `_changePage()` method accepts an int value of the selected index. The `selectedIndex` is used with the `setState()` method to set the `_currentIndex` and `_currentPage` variables.

The `_currentIndex` equals the `selectedIndex`, and the `_currentPage` equals the page from the `List _listPages` that corresponds to the selected index.

It's important to note that the `Widget _currentPage` variable displays each page selected without the need for a `Navigator` widget. This is a great example of the power of customizing widgets to your needs.

```

void _changePage(int selectedIndex) {
  setState(() {
    _currentIndex = selectedIndex;
    _currentPage = _listPages[selectedIndex];
  });
}

```

8. On the first line after the `_HomeState` class definition, add the variables `_currentIndex`, `_listPages`, and `_currentPage`. The `_listPages` List holds each page's Class name.

```

int _currentIndex = 0;
List _listPages = List();
Widget _currentPage;

```

9. Override `initState()` to add each page to the `_listPages` List and initialize the `_currentPage` with the `Birthdays()` page. Note the use of cascade notation; the double dots allow you to perform a sequence of operations on the same object.

```

@override
void initState() {
  super.initState();

  _listPages
    ..add(Birthdays())
    ..add(Gratitude())
    ..add(Reminders());
  _currentPage = Birthdays();
}

```

10. Add to the top of the `home.dart` file the imports for each page that will be created next.

```
import 'package:flutter/material.dart';
import 'gratitude.dart';
import 'reminders.dart';
import 'birthdays.dart';
```

Here's the entire `home.dart` file:

```
import 'package:flutter/material.dart';
import 'gratitude.dart';
import 'reminders.dart';
import 'birthdays.dart';

class Home extends StatefulWidget {
  @override
  _HomeState createState() => _HomeState();
}

class _HomeState extends State<Home> {
  int _currentIndex = 0;
  List _listPages = List();
  Widget _currentPage;

  @override
  void initState() {
    super.initState();

    _listPages
      ..add(Birthdays())
      ..add(Gratitude())
      ..add(Reminders());
    _currentPage = Birthdays();
  }

  void _changePage(int selectedIndex) {
    setState(() {
      _currentIndex = selectedIndex;
      _currentPage = _listPages[selectedIndex];
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('BottomNavigationBar'),
      ),
      body: SafeArea(
        child: Padding(
          padding: EdgeInsets.all(16.0),
          child: _currentPage,
        ),
      ),
    ),
  ),
}
```

```

        bottomNavigationBar: BottomNavigationBar(
          currentIndex: _currentIndex,
          items: [
            BottomNavigationBarItem(
              icon: Icon(Icons.cake),
              title: Text('Birthdays'),
            ),
            BottomNavigationBarItem(
              icon: Icon(Icons.sentiment_satisfied),
              title: Text('Gratitude'),
            ),
            BottomNavigationBarItem(
              icon: Icon(Icons.access_alarm),
              title: Text('Reminders'),
            ),
          ],
          onTap: (selectedIndex) => _changePage(selectedIndex),
        ),
      );
    }
  }
}

```

- 11. Create three StatelessWidget pages and call them Birthdays, Gratitude, and Reminders. Each page will have a Scaffold with Center() for the body. The Center child is an Icon with a size value of 120.0 pixels and a color property. Here are the three Dart files, birthdays.dart, gratitude.dart, and reminders.dart:**

```

// birthdays.dart
import 'package:flutter/material.dart';

class Birthdays extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.cake,
          size: 120.0,
          color: Colors.orange,
        ),
      ),
    );
  }
}

// gratitude.dart
import 'package:flutter/material.dart';

class Gratitude extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.sentiment_satisfied,

```

```

        size: 120.0,
        color: Colors.lightGreen,
      ),
    ),
  );
}

// reminders.dart
import 'package:flutter/material.dart';

class Reminders extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Icon(
          Icons.access_alarm,
          size: 120.0,
          color: Colors.purple,
        ),
      ),
    );
  }
}

```



How It Works

The `BottomNavigationBar` `items` property has a List of three `BottomNavigationBarItems`. For each `BottomNavigationBarItem`, you set an `icon` property and a `title` property. The `Bottom-`

NavigationBar onTap passes the selected index value to the `_changePage` method. The `_changePage` method uses `setState()` to set the `_currentIndex` and `_currentPage` to display. The `_currentIndex` sets the selected `BottomNavigationBarItem`, and the `_currentPage` sets the current page to display from the `_listPages` List.

USING THE BOTTOMAPPBAR

The `BottomAppBar` widget behaves similarly to the `BottomNavigationBar`, but it has an optional notch along the top. By adding a `FloatingActionButton` and enabling the notch, the notch provides a nice 3D effect so it looks like the button is recessed into the navigation bar (Figure 8.2). For example, to enable the notch, you set the `BottomAppBar` shape property to a `NotchedShape` class like the `CircularNotchedRectangle()` class and set the `Scaffold` `floatingActionButtonLocation` property to `FloatingActionButtonLocation.endDocked` or `centerDocked`. Add to the `Scaffold` `floatingActionButton` property a `FloatingActionButton` widget, and the result shows the `FloatingActionButton` embedded into the `BottomAppBar` widget, which is the notch.



FIGURE 8.2: `BottomAppBar` with embedded `FloatingActionButton` creating a notch

```
BottomAppBar(  
  shape: CircularNotchedRectangle(),  
  
)  
  
floatingActionButtonLocation: FloatingActionButtonLocation.endDocked,  
floatingActionButton: FloatingActionButton(  
  child: Icon(Icons.add),  
),
```

TRY IT OUT Creating the BottomAppBar App

In this example, the `BottomAppBar` has a `Row` as a child with three `IconButton`s to show selection items. The main objective is to use a `FloatingActionButton` to dock it to the `BottomAppBar` with a notch. The notch is enabled by the `BottomAppBar` shape property set to `CircularNotchedRectangle()`.

1. Create a new Flutter project and name it `ch8_bottom_app_bar`. Again, you can follow the instructions in Chapter 4. For this project, you only need to create the `pages` folder.
2. Open the `home.dart` file, and add to the body a `SafeArea` with a `Container` as a child.

```
body: SafeArea(  
  child: Container(),  
),
```

3. Add a `BottomAppBar()` widget to the `Scaffold` `bottomNavigationBar` property.

```
bottomNavigationBar: BottomAppBar(),
```

4. To enable the notch, you set two properties.

- **First set the `BottomAppBar` shape property to `CircularNotchedRectangle()`. Set the color property to `Colors.blue.shade200` and add a `Row` as a child.**
- **Next set `floatingActionButtonLocation`, which you will handle in step 7.**

```
bottomNavigationBar: BottomAppBar(  
  color: Colors.blue.shade200,  
  shape: CircularNotchedRectangle(),  
  child: Row(),  
)
```

5. Continue by adding to the `Row` a `mainAxisAlignment` property as `MainAxisAlignment.spaceAround`. The `spaceAround` constant allows the `IconButton`s to have even spacing between them.

```
bottomNavigationBar: BottomAppBar(  
  color: Colors.blue.shade200,  
  shape: CircularNotchedRectangle(),  
  child: Row(  
    mainAxisAlignment: MainAxisAlignment.spaceAround,  
    children: <Widget>[  
    ],  
  ),  
)
```

6. Add three `IconButton`s to the `Row` children list. After the last `IconButton`, add a `Divider()` to add an even space to the right since `FloatingActionButton` is docked on the right side of the `BottomAppBar`. Instead of a `Divider()`, you could have used a `Container` with a width property.

```
bottomNavigationBar: BottomAppBar(  
  color: Colors.blue.shade200,  
  shape: CircularNotchedRectangle(),  
  child: Row(  
    mainAxisAlignment: MainAxisAlignment.spaceAround,  
    children: <Widget>[  
      IconButton(  
        icon: Icon(Icons.access_alarm),  
        color: Colors.white,  
        onPressed: () {},  
      ),  
      IconButton(  
        icon: Icon(Icons.bookmark_border),  
        color: Colors.white,  
        onPressed: () {},  
      ),  
      IconButton(  
        icon: Icon(Icons.flight),  
        color: Colors.white,  
        onPressed: () {},  
      ),  
      Divider(),  
    ],  
  ),  
)
```

- 7. Set the notch location of the `floatingActionButtonLocation` property to `FloatingActionButtonLocation.endDocked`. You could also set it to `centerDocked`.**

```
floatingActionButtonLocation: FloatingActionButtonLocation.endDocked,
```

- 8. Add a `FloatingActionButton` to the `floatingActionButton` property.**

```
floatingActionButton: FloatingActionButton(  
  backgroundColor: Colors.blue.shade200,  
  onPressed: () {},  
  child: Icon(Icons.add),  
),
```

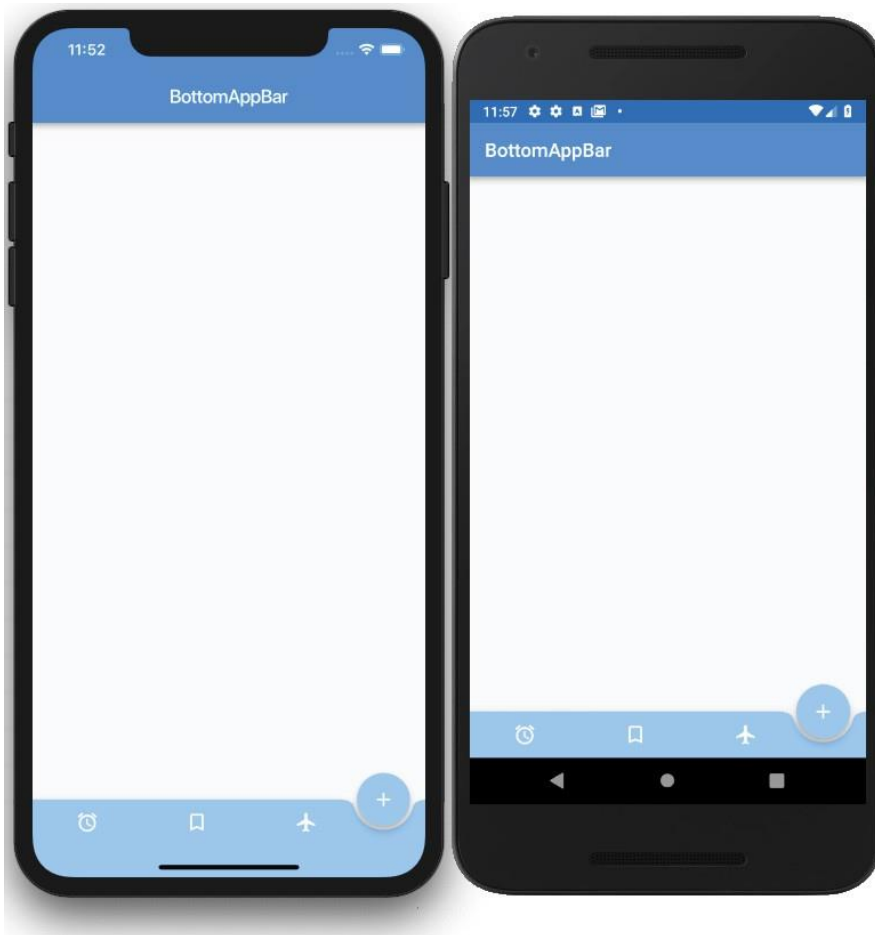
Here's the entire `home.dart` file source code:

```
import 'package:flutter/material.dart';  
  
class Home extends StatefulWidget {  
  @override  
  _HomeState createState() => _HomeState();  
}  
  
class _HomeState extends State<Home> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('BottomAppBar'),  
      ),  
      body: SafeArea(  
        child: Container(),  
      ),  
      bottomNavigationBar: BottomAppBar(  
        color: Colors.blue.shade200,  
        shape: CircularNotchedRectangle(),  
        child: Row(  
          mainAxisAlignment: MainAxisAlignment.spaceAround,  
          children: <Widget>[  
            IconButton(  
              icon: Icon(Icons.access_alarm),  
              color: Colors.white,  
              onPressed: () {},  
            ),  
            IconButton(  
              icon: Icon(Icons.bookmark_border),  
              color: Colors.white,  
              onPressed: () {},  
            ),  
            IconButton(  
              icon: Icon(Icons.flight),  
              color: Colors.white,  
              onPressed: () {},  
            ),  
            Divider(),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

```

    ),
  ),
  floatingActionButtonLocation: FloatingActionButtonLocation.endDocked,
  floatingActionButton: FloatingActionButton(
    backgroundColor: Colors.blue.shade200,
    onPressed: () {},
    child: Icon(Icons.add),
  ),
);
}
}

```



How It Works

To enable the notch, two properties need to be set for the `Scaffold` widget. The first is to use a `BottomAppBar` with the `shape` property set to `CircularNotchedRectangle()`. The second is to set the `floatingActionButtonLocation` property to `FloatingActionButtonLocation.endDocked` or `centerDocked`.

Solution:

Hero animation:

```
1  import 'package:flutter/material.dart';
2
3  class HeroActionCard extends StatelessWidget {
4    final String tag;
5    final String title;
6    final String subtitle;
7    final IconData icon;
8    final List<Color> gradient;
9    final VoidCallback onTap;
10
11    const HeroActionCard({
12      super.key,
13      required this.tag,
14      required this.title,
15      required this.subtitle,
16      required this.icon,
17      required this.gradient,
18      required this.onTap,
19    });
20
21    @override
22    Widget build(BuildContext context) {
23      return Hero(
24        tag: tag,
25        flightShuttleBuilder: (context, animation, direction, from, to) {
26          return Material(
27            color: Colors.transparent,
28            child: to.widget,
29          ); // Material
30        },
31        child: Material(
32          color: Colors.transparent,
```

```

33   child: InkWell(
34     borderRadius: BorderRadius.circular(15),
35     onTap: onTap,
36     child: Container(
37       height: 120,
38       padding: const EdgeInsets.all(15),
39       decoration: BoxDecoration(
40         borderRadius: BorderRadius.circular(15),
41         gradient: LinearGradient(colors: gradient),
42       ), // BoxDecoration
43       child: Column(
44         crossAxisAlignment: CrossAxisAlignment.start,
45         children: [
46           Icon(icon, color: Colors.white),
47           const Spacer(),
48           Text(title,
49             style: const TextStyle(
50               color: Colors.white, fontWeight: FontWeight.bold)), // TextStyle // Text
51           Text(subtitle,
52             style:
53               const TextStyle(color: Colors.white70, fontSize: 12)), // Text
54         ],
55       ), // Column
56     ), // Container
57   ), // InkWell
58 ), // Material
59 ); // Hero
60 }
61 }

```

```

1  import 'package:flutter/material.dart';
2  import 'package:cloud_firestore/cloud_firestore.dart';
3  import 'package:firebase_auth/firebase_auth.dart';
4  import '../services/room_service.dart';
5  import 'create_room_screen.dart';
6  import 'join_room_screen.dart';
7  import '../widgets/hero_action_card.dart';
8

```

```
// ===== HERO ACTIONS =====
```

```
Row(  
  children: [  
    Expanded(  
      child: HeroActionCard(  
        tag: "create_room",  
        title: "Create Room",  
        subtitle: "Start a new space",  
        icon: Icons.grid_view_rounded,  
        gradient: [Colors.blue, Colors.purple],  
        onTap: () {  
          Navigator.push(  
            context,  
            PageRouteBuilder(  
              transitionDuration:  
                const Duration(milliseconds: 700),  
              pageBuilder: (_, __, ___) =>  
                const CreateRoomScreen(),  
            ), // PageRouteBuilder  
          );  
        },  
      ), // HeroActionCard  
    ), // Expanded  
    const SizedBox(width: 12),  
    Expanded(  
      child: HeroActionCard(  
        tag: "join_room",  
        title: "Join Room",  
        subtitle: "Enter with code",  
        icon: Icons.login,  
        gradient: [Colors.green, Colors.teal],
```

```
        onTap: () {  
          Navigator.push(  
            context,  
            PageRouteBuilder(  
              transitionDuration:  
                const Duration(milliseconds: 700),  
              pageBuilder: (_, __, ___) => const JoinRoomScreen(),  
            ), // PageRouteBuilder  
          );  
        },  
      ), // HeroActionCard  
    ), // Expanded  
  ],  
), // Row
```

BottomNavBar:

```
child: BottomNavigationBar(  
  currentIndex: currentIndex,  
  type: BottomNavigationBarType.fixed,  
  backgroundColor: Colors.transparent,  
  elevation: 0,  
  selectedItemColor: Colors.blueAccent,  
  unselectedItemColor: Colors.grey,  
  selectedLabelStyle: const TextStyle(fontSize: 12),  
  unselectedLabelStyle: const TextStyle(fontSize: 11),  
  onTap: (index) {  
    setState(() {  
      currentIndex = index;  
    });  
  },  
  items: const [  
    BottomNavigationBarItem(  
      icon: Icon(Icons.home_outlined),  
      activeIcon: Icon(Icons.home),  
      label: "Home",  
    ), // BottomNavigationBarItem  
    BottomNavigationBarItem(  
      icon: Icon(Icons.people_outline),  
      activeIcon: Icon(Icons.people),  
      label: "Friends",  
    ), // BottomNavigationBarItem  
    BottomNavigationBarItem(  
      icon: Icon(Icons.person_outline),  
      activeIcon: Icon(Icons.person),  
      label: "Profile",  
    ), // BottomNavigationBarItem  
    BottomNavigationBarItem(  
      icon: Icon(Icons.settings_outlined),  
      activeIcon: Icon(Icons.settings),  
      label: "Settings",  
    ), // BottomNavigationBarItem  
  ],  
), // BottomNavigationBar
```

Navigator:


```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(builder: (_) => const AvatarScreen()),  
);
```

```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const LoginScreen(),  
  ), // MaterialPageRoute  
);
```

```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const DashboardScreen(),  
  ), // MaterialPageRoute  
);
```

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const SignupScreen(),  
  ), // MaterialPageRoute  
);
```

```
Navigator.push(  
  context,  
  PageRouteBuilder(  
    transitionDuration:  
      const Duration(milliseconds: 700),  
    pageBuilder: (_, __, ___) => const JoinRoomScreen(),  
  ), // PageRouteBuilder  
);
```

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const JoinRoomScreen(),  
  ), // MaterialPageRoute  
);
```

